ONE-TAKE, CUTIE

The All-In-One Programming Skills Evaluation Guide

A project by the

CCSS Research & Development Unit

Prepared by

Chazz C. Manubay

CCSS R&D Team Leader

First Edition, December 2023

Foreword

> "Let's go invent tomorrow instead of worrying about what happened yesterday." – Steve Jobs

The Programming Skills Evaluation (PSE; popularly known as the Battery Examination), is a significant assessment held in the UE College of Computer Studies and Systems (UE-CCSS), pioneered by the late former UE-CCSS dean, Professor Rodany Arcon Merida. It aims to assess the students' applied knowledge on web development and programming, critical thinking, and their capability to manage their workflow in a set span of time.

As a supplementary resource, the UE-CCSS R&D Unit, in collaboration with the administration of the UE-CCSS, came up with this guide to help the students manage their workflow during the evaluation, and to give them a general collection of information that they may use in the systems they are required to develop.

This guide is mainly targeted towards students who will be using the PHP programming language as the back-end language of the system that they will be developing. With that, this guide will not be covering the scope of the problem sets given to BSEMC-GD, as well as the Designing Skills Evaluation of the BSEMC-DA.

This guide will also not be covering the transaction module of the PSE. It is because this module is different for each group. Instead, this module is directed towards helping you establish the database and the CRUD module, which are two vital components for the transaction module to function.

It is important to keep in mind that this guide is **strictly for PSE purposes only** and is **not for commercial distribution or sale**.

Any violation of these terms and conditions may result in legal consequences sanctioned by the UE-CCSS Administration.

Table of Contents

Foreword i
Table of Contents ii

Chapter 1: Setting Up Your Workspace 1
Chapter 2: Getting Started 2
Chapter 3: Establishing the Database 4
Creating the Database 4
Creating a Database Table 5
Chapter 4: Connecting to the Database 7
Chapter 5: Creating the Index Page 8
Chapter 6: Creating the Add Module 11
Chapter 7: Creating the Edit Module 14
Chapter 8: Creating the Delete Module 16
Chapter 9: String Manipulation 17
Combining/Concatenating Strings 17
String Length 17
Substring 17
Converting Strings to Upper/Lower Case 17
Trim Whitespaces 17
Date Formatting 18
Chapter 10: Additional Resources 19

Chapter 1: Setting Up Your Workspace

Recommended Time Allotment: 15 minutes or less.

First things first, you must ensure that you have the following software ready for use:

- **Integrated Development Environment (IDE)**: While Visual Studio Code is widely used, you may try to request from the Laboratory Technician on-duty if you can install another IDE.
  R&D Recommendation: Use VS Code to avoid unnecessary issues that may arise, as it is available in every computer laboratory.

- **XAMPP**: This is the Apache distribution used to run your PHP system, as it has an integrated support for MySQL.

- **Web Browser**: This is where you will be able to view the system that you are developing, as well as the PhpMyAdmin module of XAMPP. You may choose to use either Microsoft Edge, Firefox (if available), or Google Chrome.
  R&D Recommendation: Stick with Google Chrome.

During the PSE, you will also be asked to fill out an attendance sheet. Therefore, we strongly recommend that you **prepare your writing instruments** (pen, highlighter) with you before you stow away your bags at the shelves.

We also strongly urge you to **place any electronic and storage devices inside your bags or on the shelves** during the PSE to avoid any issues and consequences during your examination. Instead, focus on building up your printed materials until you feel confident that you will be able to develop the system by just using those materials.

Chapter 2: Getting Started

Recommended Time Allotment: 15 minutes or less.

The moment you enter the computer laboratory assigned to you, immediately prepare all the items stated in Chapter 1. We have created a small checklist for you so that you can check if you already have prepared the necessary items:

- ☐ IDE (VS Code)
- ☐ XAMPP
- ☐ Web Browser (Google Chrome)
- ☐ Writing Instruments
- ☐ Printed Materials

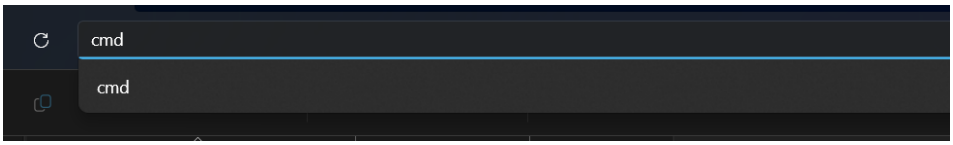
Afterwards, we encourage you to proceed to the **htdocs** folder located inside of the **xampp** folder (usually found in the C Drive) and create your project folder.

> **TIP**: Don't know what folder name to use? We recommend using ***SURNAME_PSE*** (e.g. MANUBAY_PSE, DELA_CRUZ_PSE)

After creating your project folder, start up the Apache and MySQL service in XAMPP. As a shortcut to opening your project folder in VS Code, you may do the following:

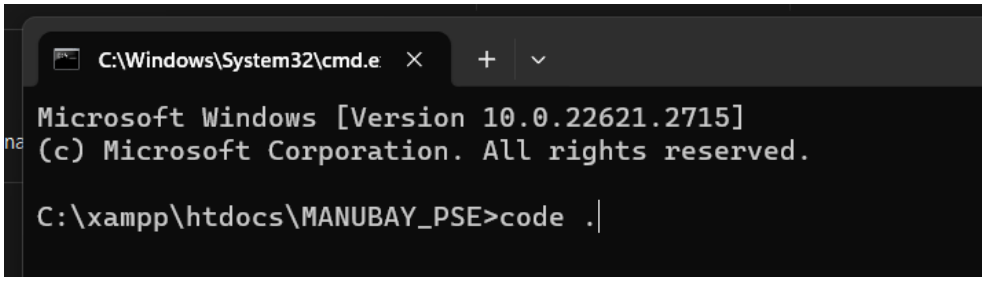
1. Click on the URL bar of the folder window.

2. Type **cmd**.

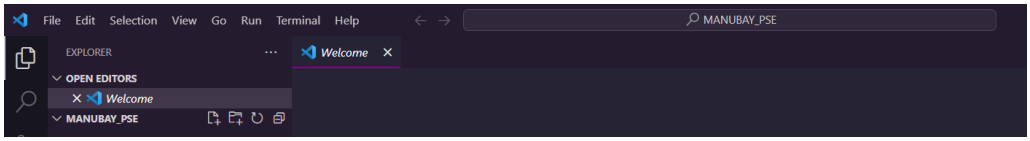


3. Press ENTER, and a Command Prompt window must open.

4. Type **code .** and press ENTER.



5. This will automatically open VS Code and show the project folder you have created.



Once you have opened your IDE, and have started Apache and MySQL in XAMPP, you may now proceed to open your web browser.

Ideally, you must always open at least two (2) browser tabs: a tab for your project, and a tab for PhpMyAdmin.

To access your project, you may enter the following URL:

localhost:8080/FOLDER_NAME

To access PhpMyAdmin, you may enter the following URL:

localhost:8080/phpMyAdmin

Once you have opened these two browser tabs, you may now proceed with Chapter 3.

Chapter Checklist:

- ☐ I have finished creating my project folder.
- ☐ I have started the Apache and MySQL services.
- ☐ I have opened my project folder in VS Code.
- ☐ I have opened my project and PhpMyAdmin on the browser.

Chapter 3: Establishing the Database

Recommended Time Allotment: 15 minutes or less.

Congratulations! You have finished setting up your workspace and you are now ready to create the **most crucial part** of PSE: establishing your database.

Note: This chapter and the next chapters require knowledge of basic SQL database, as well as PHP database connection.

Creating the Database

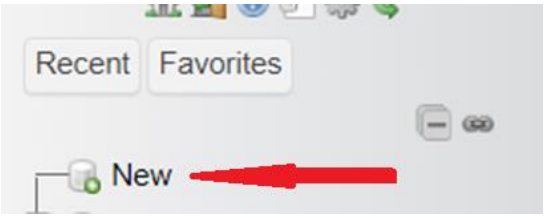
To begin, proceed to the PhpMyAdmin module and create a database. You may also follow the naming convention of your folder name – only this time, it's in lowercase. So, if your project folder name is DELA_CRUZ_PSE, your database name will be dela_cruz_pse.

You may choose between two ways of creating a database: (1) using the PhpMyAdmin's features or (2) writing an SQL query.

Method 1: Using PhpMyAdmin's Features

PhpMyAdmin offers a user interface that allows you to establish your database without having to write SQL queries. However, there are multiple features offered by PhpMyAdmin, which may intimidate some developers. But we really recommend that you do not think too deeply about how much it can offer, as you only need one feature at a time.

To establish your database through PhpMyAdmin's features, you can click on the "New" button found at the leftmost side of the PhpMyAdmin module.

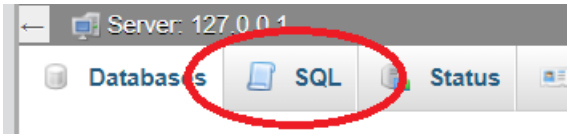


A field then will be shown where you can enter your database's name.

After you click the "Create" button, you have successfully created your database.

Method 2: Writing an SQL Query

If you feel comfortable with writing queries instead of navigating through PhpMyAdmin's features, you can proceed to the SQL tab found at the top part of the main panel of PhpMyAdmin.



You may then proceed writing the query to create a database:

```
CREATE DATABASE database_name;
```

After writing the query, you can click the "Go" button located just under the SQL text editor. After the query runs, you have successfully created your database using SQL query.

Creating a Database Table

Just like establishing the database, you can choose between two (2) ways of creating database tables: using the PhpMyAdmin's features or writing an SQL query. We feel like the PhpMyAdmin's table creation features have a lot of information in them – some not fully covered in our subjects. That is why we recommend that you instead write an SQL query. To create a table, you can follow these general formats:

Table with auto-incrementing primary key:

```
CREATE TABLE tables (
    table_id INT NOT NULL AUTO_INCREMENT,
    table_field VARCHAR(255) NOT NULL,
    another_field DECIMAL(19,4) NOT NULL,
    PRIMARY KEY (table_id)
);
```

Table with system-generated primary key:

```
CREATE TABLE tables (
    table_id VARCHAR(255) NOT NULL,
    first_field VARCHAR(255) NOT NULL,
    second_field INT NOT NULL,
    third_field DECIMAL(19,4) NOT NULL,
    PRIMARY KEY (table_id)
);
```

A rule of thumb to determine whether you need a table with an auto-incrementing primary key, or a table with a system-generated primary key, is to look for **context clues** inside the problem set.

Chapter Checklist:

- ☐ I have created my database.
- ☐ I have created my database tables.

Chapter 4: Connecting to the Database

Recommended Time Allotment: 5 minutes or less.

Congratulations for making it this far! You are almost done with all the preparations before you get into writing all the codes that you need to pass the battery exam.

Don't worry, the things you have done in the previous chapters are highly significant in making sure that the next chapters work. And for this chapter, we will establish a connection between the database you just created and the project that you are about to create.

To start, you can go back to your IDE and create a PHP file that will contain all the codes to connect you to your database. Generally, we would name this "connect.php".

Now, you can enter the following code snippet that will establish your connection to the database:

```
<?php
$connection = new mysqli('localhost', 'root', '', 'database_name');
if (!$connection)
{
    echo "Connection has failed.";
    die(mysqli_error($connection));
}
```

You might be wondering about why there is no "?>" at the end of the code. That's because since we're only doing PHP for the entire file, we no longer need to enclose the PHP. You can, but it's best practice that you don't. 😉

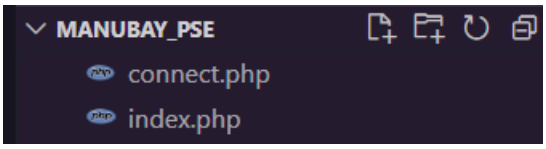
Chapter 5: Creating the Index Page

Recommended Time Allotment: 30 minutes or less.

You have successfully created a connection between your project and the database. Give yourself a pat at the back for making it this far.

Now we will start with one of the CRUD modules that you need to accomplish for the battery exam: the READ module. In this scenario, we will be calling it the "Index" module, as it's a good user experience to immediately see all the information (and it's intuitive as well).

To start, create your "index.php" file. After doing so, your files should look like this:



Open the "index.php" file, and then write your HTML code first.

> **TIP**: To immediately write the HTML boilerplate (DOCTYPE, head, body, etc.), you can use the "!" emmet. Emmets are shortcut keys that you can use to speed up your code-writing process. For this case, just type "!" (excluding the quotation marks) and then press TAB.

After creating the boilerplate, go to the topmost part of your file and create a PHP scope. This time, the "?>" end tag is important. First write an include statement for the previously created "connect.php" file. This will allow you to use the "$connection" variable that we defined in that file.

```
1  <?php
2
3  include "connect.php";
```

After including the connection file, let’s proceed with the logic of searching and listing all your data.

We must consider if a search has been requested or not. To do this, we must write the following code snippet:

```
if (isset($_GET['search']) && strlen($_GET['search']) > 0) {

    // The condition above checks if a GET request for search has been
    // submitted and if the request is not blank.

    $search = trim(htmlspecialchars(strip_tags($_GET['search'])));

    // You may be wondering why there is so much functions called in
    // the $search variable. That is because we want to sanitize user
    // input before the system processes it, to avoid XSS and other
    // forms of intrusion.

    $query = "SELECT * FROM table_name WHERE table_id LIKE '%$search%'
             OR table_field LIKE '%$search%'";

    // For this query, we are considering any content of table_id and
    // table_field to match the search keyword submitted by the user.

} else {

    $query = "SELECT * FROM table_name";

    // If there is no search keyword being submitted, this query will
    // show all the data inside table_name.

}

$result = mysqli_query($connection, $query);
```

To show the data of your database, we recommend that you use HTML tables. Since we’re short on time, you can forget about making it fancy. But of course, making it fancy is a plus if it makes you feel better (and if you have enough time to do so).

We recommend that you follow this general template for making your search bar and data tables:

```
<a href='create.php'>Add</a>

<!-- SEARCH BAR -->

<form action='index.php' method='GET'>
    <input placeholder='Search...' type='text' name='search'
        id='search' value='<?= $search ?>'>
    <button type='submit'>Search</button>
</form>

<table>
    <thead>
        <th>ID</th>
        <th>Field Name</th>
        <th>Actions</th>
    </thead>
    <tbody>

        <?php
        if ($result) {
            while($row = mysqli_fetch_assoc($result)) {
        ?>

        <tr>
            <td><?= $row['table_id'] ?></td>
            <td><?= $row['table_field'] ?></td>
            <td>
                <a href='/edit.php?id=<?=$row['table_id']?>'>Edit</a>
                <form action='delete.php' method='POST'>
                    <input type='hidden' value='<?=$row['table_id']?>'
                        name='id'>
                    <button type='submit'>Delete</button>
                </form>
            </td>
        </tr>

        <?php
            }
        }
        ?>
    </tbody>
</table>
```

Chapter 6: Creating the Add Module

Recommended Time Allotment: 35 minutes or less.

Congratulations on finishing the READ module! We're now ready to move on to the second module of the CRUD modules: the CREATE module. To start, create a file called "create.php". You may also start writing the boilerplate and designing the form that you will be using.

You can observe the following general guidelines for creating the form:

- The method of the form must be in POST.
- The 'name' and 'type' attributes of the <input> tag is vital for the retrieval of information.
- The 'id' attribute of the <input> tag is used in the 'for' attribute of the <label> tag.
- Use the <button> tag with the 'type' attribute set as 'submit'.
- If you want to style your form, we recommend that you separate each set of <label> and <input> into their own tags.

Once you have established your form, you can proceed to the topmost part of your file and create a PHP scope. Here is a general code snippet on how you can process the form submission:

```
<?php

include "connect.php";

$errors = [];

$text_field = null;
$date_field = null;
$number_field = null;
```

```php
$money_field = null;
if (isset($_POST['create'])) {
    // The $_POST['create'] variable came from the 'name'
    // attribute specified in the submit <button> tag.

    $text_field = trim(htmlspecialchars(
        strip_tags($_POST['text-field']));
    $date_field = $_POST['date-field'];
    $number_field = $_POST['number-field'];
    $money_field = $_POST['money-field'];

    // The variables indicated inside the $_POST[] came from
    // the name attributes of the inputs in the form.

    // Validation of Text

    if (empty($text_field)) {
        $errors['text-field'] = 'Please enter an input.';
    }

    // Validation of Integer

    if (!is_int(intval($number_field))) {
        $errors['number-field'] = 'Please enter
            a valid number.';
    }

    // Validation of Money

    $money_pattern = '/^-?\d+(?:\.\d{1,2})?$/';

    if ($money_field < 0) {
        $errors['money-field'] = 'Money cannot be negative.';
    } else if (!preg_match($money_pattern, $money_field) {
        $errors['money-field'] = 'Please enter a valid money
            amount.';
    }
```

```
    // Validation of Date

    $date_timestamp = strtotime($date_field);


    if (!$date_timestamp) {
        $errors['date-field'] = 'Please enter a valid date.';
    } else if ($date_timestamp < time()) {
        $errors['date-field'] = 'Date cannot be in the past.';
    }

    if (count($errors) == 0) {
        $date_format = new DateTime($date_field);
        $date_field = $date_format->format('Y-m-d h:i:s');

        $query = "INSERT INTO table_name (text_field,
            number_field, money_field, date_field) VALUES
            ('$text_field', '$number_field', '$money_field',
            '$date_field');
        if ($connection->query($query)) {
            $connection->close();
            header("Location: index.php");
        } else {
            echo "Error: " . $connection->error;
        }
    }
}

?>
```

The code snippet above shows different ways to validate user input, which may come in handy depending on the situation or the problem set. You may add or change the validations based on what is required from you.

Likewise, the query above is very simple. We recommend that you use **prepared statements** to secure your queries.

Chapter 7: Creating the Edit Module

Recommended Time Allotment: 35 minutes or less.

Congratulations for making it this far! So far, we have completed two of the CRUD modules, and we are halfway finishing the first part of the PSE. The next module that we will be making is the UPDATE module, otherwise known as the Edit module.

To start, let's create a file called "edit.php". To make things faster, we recommend that you copy the layout of the form in the CREATE module into this file and tweak it. Here is a sample of how you can get the information of a specific record from the database and place it inside the form inputs:

```
$text_field = null;
$number_field = null;
$date_field = null;

if (!isset($_POST['edit'])) {

    $id = trim(htmlspecialchars(strip_tags(
        $_GET['asset_id']))));

    $query = "SELECT * FROM table_name WHERE table_id = '$id'";
    $result = mysqli_query($connection, $query);

    if (mysqli_num_rows($result) == 1) {
        $row = mysqli_fetch_assoc($result);
        $id = $row['table_id'];
        $text_field = $row['text_field'];
        $number_field = intval($row['number_field']);
        $date_field = date_format(date_create(
            $row['date_field']), "Y-m-d\TH:i");
    } else {
        echo "ID not found.";
        die();
    }
}
```

After specifying this at the top part of your file, you can retrieve the variables' values by setting the 'value' attribute of each respective form input. For example:

```
<div>
    <label for='text-field'>Text Field</label>
    <input type='text' name='text-field' id='text-field'
        value='<?= $text_field ?>'>
</div>
```

You can then specify the logic for updating the record in the else block:

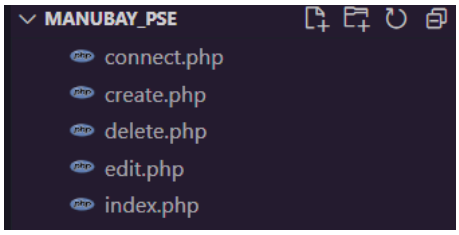
```
if (!isset($_POST['edit'])) {
    // ...
} else {
    // Validate inputs here
    // ...

    if (count($errors) == 0) {
        $date_format = new DateTime($date_field);
        $date_field = $date_format->format('Y-m-d h:i:s');

        $query = "UPDATE table_name SET text_field =
                 '$text_field', number_field = '$number_field',
                 date_field = '$date_field' WHERE
                 table_id = $id";
        if ($connection->query($query)) {
            $connection->close();
            header("Location: index.php");
        } else {
            echo "ID not found.";
            die();
        }
    }
}
```

Chapter 8: Creating the Delete Module

Recommended Time Allotment: 15 minutes or less.

We have finally reached the last module of the CRUD modules. This module is much simpler than the previous ones, as it does not need any form or input validation.

To start, create a “delete.php” file. So far, your files should look like this:



Go into the “delete.php” file and enter the code needed for deleting a record. You must make sure that the form you initialized in the “index.php” for deleting a record must be in POST. Here is a general template for the DELETE functionality:

```
<?php

include "connect.php";

if(isset($_POST['delete'])) {
    $id = $_POST['id'];
    $query = "DELETE FROM table_name WHERE table_id = '$id'";
    if ($connection->query($query)) {
        $connection->close();
        header("Location: index.php");
    } else {
        echo "Unable to delete record.";
        die();
    }
}
```

With that, you have successfully finished the CRUD modules of your PSE system. You are now ready to do your transaction module.

Chapter 9: String Manipulation

We have a few tips we'd like to share with you about string manipulation in PHP:

Combining/Concatenating Strings

```
$string_one = "I will pass";
$string_two = " this take.";
$string_three = $string_one . $string_two;

echo $string_three; // Output: I will pass this take.
```

String Length

```
$text = "This take is my last take.";

$length = strlen($text); // 26
```

Substring

```
$text = "ONE TAKE CUTIE";

$first_word = substr($text, 0, 3); // ONE
$second_word = substr($text, 4, 4); // TAKE
$third_word = substr($text, 9, 5); // CUTIE
```

Converting Strings to Upper/Lower Case

```
$text = "Pasado ako dito.";

$uppercase = strtoupper($text); // PASADO AKO DITO.
$lowercase = strtolower($text); // pasado ako dito.
```

Trim Whitespaces

```
$full_text = "   PASADO AKO.   ";
$trimmed_text = trim($full_text);

echo "MANIFESTING NA " . $trimmed_text;
// Output: MANIFESTING NA PASADO AKO.
```

Date Formatting

When formatting dates, we recommend that you use the following syntax:

```
$date = $_POST['date-field'];

$date_format = date_format(date_create($date),
                "FORMAT HERE");
```

Here are some of the formatting keywords that you might need to use:

Keyword	Date Formatted	Format
d	Day in a month	01 - 31
j		1 - 31
D	Day in a week	Mon - Sun
l		Sunday - Saturday
N		1 (Mon) - 7 (Sun)
w		0 (Sun) - 6 (Sat)
m	Month	01 - 12
n		1 - 12
F		January - December
M		Jan - Dec
Y	Year	2023
y		23
L		1 (leap year) or 0 (not leap year)
a	Time Meridian	am/pm
A		AM/PM
g	Hours	1 - 12
h		01 - 12
G		0 - 23
H		00 - 23
i	Minutes	00 - 59
s	Seconds	00 - 59

You can use these keywords, for example, in the following scenarios:

```
$date = new DateTime(); // Gets date and time today

$date_format = $date->format("Y-m-d"); // 2023-12-17
$date_format = $date->format("g:i A"); // 3:40 PM
$date_format = $date->format("M j, Y"); // Dec 17, 2023
$date_format = $date->format("\I\\t \i\s G:i:s.");
    // It is 15:45:39.
```

Chapter 10: Additional Resources

Since we cannot give you everything that there is to ace your PSE, we would like to recommend some resources that we think will help you a lot:

1. W3Schools: They provide tutorials in basically everything that you will need for your PSE. Just search for their website at Google together with the programming language that you need to learn about, and they will most likely have a comprehensive tutorial for it.

   Link: https://www.w3schools.com/php/

2. WebsiteSetup's PHP Cheat Sheet: This is a PDF of all the functions that you might need in PHP. This includes array functions, string functions, date functions, and more.

   Link: https://websitesetup.org/wp-content/uploads/2020/09/PHP-Cheat-Sheet.pdf

We hope that these two resources, as well as this guide, will equip you with additional knowledge, and help you emerge victorious in the Programming Skills Evaluation. Good luck, CCSS Warrior!